

Chapter TEN - Data Transformation-Data Reduction-

We discussed in *Chapter 2* Data Transformation we will return back now:

Data transformation is a technique used to **convert** the raw data into a suitable format that efficiently eases data mining and retrieves strategic information.

Data transformation includes **data cleaning techniques** and a **data reduction technique** to convert the data into the appropriate form.

Data transformation **changes** the **format**, **structure**, or **values of the data** and converts them into **clean, usable data**.

Data may be transformed at two stages of the data pipeline for data analytics projects.

Data Transformation Techniques:

1. Data smoothing
2. Attribute construction
3. Data aggregation
4. Data normalization
5. Data discretization
6. Data generalization

1. Data Smoothing

Data smoothing is a process that is used to **remove noise** from the dataset using some algorithms.

The **noise** is removed from the data using the techniques such as **binning, regression, clustering**.

- **Binning:** This method splits the sorted data into the number of bins and smoothens the data values in each bin considering the neighborhood values around it.
- **Regression:** This method identifies the **relation** among two dependent attributes so that if we have one attribute, it can be used to predict the other attribute.
- **Clustering:** This method **groups similar data values** and form a **cluster**. The values that lie outside a cluster are known as **outliers**.

2. Attribute Construction

In the attribute construction method, the new attributes consult the existing attributes to construct a new data set that eases data mining. New attributes are created and applied to assist the mining process from the given attributes. This simplifies the original data and makes the mining more efficient.

Ex: we may have the *height* and *width* of each plot. So here, we can construct a new attribute '**area**' from attributes 'height' and 'weight' or **BMI**, using these two features.

3. Data Aggregation

Data collection or aggregation is the method of storing and presenting data in a summary format.

The data may be obtained from multiple data sources to integrate these data sources into a data analysis description.

For example, we have a data set of sales reports of an enterprise that has quarterly sales of each year. We can aggregate the data to get the enterprise's **monthly** or **annual** sales report.

4. Data Normalization

Normalizing the data refers to scaling the data values to a much smaller range such as $[-1, 1]$ or $[0.0, 1.0]$. Discussed before.

5. Data Discretization

This is a process of **converting continuous** data into **a set of data intervals**. Continuous attribute values are substituted by small interval labels. This makes the data easier to study and analyze.

Data discretization can be **classified** into **two** types: **supervised discretization**, where the class information is used, and **unsupervised discretization**, which is based on which direction the process proceeds, i.e., 'top-down splitting strategy' or 'bottom-up merging strategy'.

For example, the values for the **age** attribute can be **replaced by the interval labels** such as (0-10, 11-20...) or (kid, youth, adult, senior).

6. Data Generalization

It converts **low-level** data attributes to **high-level data attributes** using concept **hierarchy**. This conversion from a lower level to a higher conceptual level is useful to get a clearer picture of the data.

Data generalization can be divided into two approaches:

- Data cube process (OLAP) approach.
- Attribute-oriented induction (AOI) approach.

For example, age data can be in the form of (20, 30) in a dataset. It is transformed into a higher conceptual level into a categorical value (young, old).

Data Reduction

Data reduction techniques ensure the integrity of data while reducing the data. Data reduction is a process that reduces the volume of original data and represents it in a much smaller volume.

Techniques of Data Reduction:

1. Dimensionality reduction
2. Numerosity reduction
3. Data cube aggregation
4. Data compression
5. Discretization operation

1. Dimensionality Reduction

Whenever we encounter weakly important data, we use the attribute required for our analysis.

Dimensionality reduction eliminates the attributes from the data set under consideration, thereby reducing the volume of original data.

When working with machine learning models:

datasets with **too many features** can cause issues like **slow computation** and **overfitting**. Dimensionality reduction helps by reducing the number of features while retaining key information.

Example:

*when you are **building a model to predict house prices** with **features** like **bedrooms**, **square footage**, and **location**. If you add too many features, such as room condition or flooring type, the dataset becomes large and complex.*

With too many features, training can slow down and the model may focus on irrelevant details, like flooring type, which could lead to inaccurate predictions.

Categories of Dimensionality Reduction Techniques

- Feature Selection
- Filter Methods
- Wrapper Methods
- Embedded Methods
- Feature Extraction
- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Uniform Manifold Approximation and Projection (UMAP)

Here are **three** methods of **dimensionality reduction**.

- **Wavelet Transform:** In the wavelet transform, suppose a data vector A is transformed into a numerically different data vector A' such that both A and A' vectors are of the same length.

- **Principal Component Analysis (PCA):** Suppose we have a data set to be analyzed that has tuples with n attributes. The principal component analysis identifies **k** independent tuples with n attributes that can represent the data set.

Principal Component Analysis (PCA) help in identifying and removing irrelevant or redundant features.

- **Attribute Subset Selection:** The large data set has many attributes, some of which are irrelevant to data mining or some are redundant. The core attribute subset selection reduces the data volume and dimensionality.

2. Numerosity Reduction

The numerosity reduction reduces the original data volume and represents it in a much smaller form. This technique includes **two types** **parametric** and **non-parametric** numerosity reduction.

- i) **Parametric:** Parametric numerosity reduction incorporates storing only data parameters instead of the original data. One method of parametric numerosity reduction is the regression and log-linear method.

- o **Regression and Log-Linear:** Linear regression models a relationship between the two

attributes by modeling a linear equation to the data set. Suppose we need to model a linear function between two attributes.

$$y=wx+b$$

Here, y is the response attribute, and x is the predictor attribute

Log-linear model discovers the relation between two or more discrete attributes in the database.

Suppose we have a set of tuples presented in n-dimensional space. Then the log-linear model is used to study the probability of each tuple in a multidimensional space.

ii) Non-Parametric: A non-parametric numerosity reduction technique does not assume any model. The non-Parametric technique results in a more uniform reduction, irrespective of data size, but it may not achieve a high volume of data reduction like the parametric.

Non-Parametric data reduction techniques, **Histogram**, **Clustering**, **Sampling**, **Data Cube Aggregation**, and **Data Compression**.

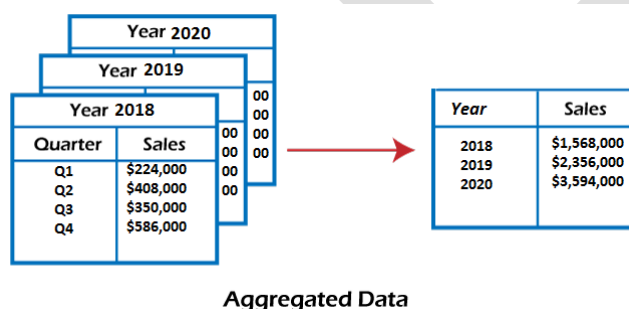
- **Histogram:** A histogram is a graph that represents frequency distribution which describes how often a value appears in the data. Histogram uses the binning method to represent an attribute's data distribution. It uses a disjoint subset which we call bin or buckets.
- **Clustering:** Clustering techniques groups similar objects from the data so that the objects in a cluster are similar to each other, but they are dissimilar to objects in another cluster.
- **Sampling:** One of the methods used for data reduction is sampling, as it can reduce the large data set into a much smaller data sample.
- **Cluster sample:** The tuples in data set D are clustered into M mutually disjoint subsets.

The data reduction can be applied by implementing SRSWOR on these clusters. A simple random sample of size s could be generated from these clusters where $s < M$.

- **Stratified sample:** The large data set D is partitioned into mutually disjoint sets called 'strata'. A simple random sample is taken from each stratum to get stratified data. This method is effective for skewed data.

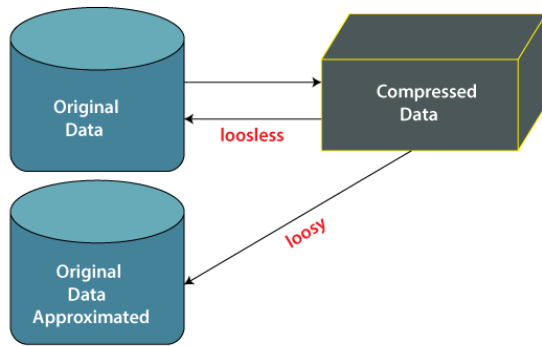
3. Data Cube Aggregation

This technique is used to aggregate data in a simpler form. Data Cube Aggregation is a multidimensional aggregation that uses aggregation at various levels of a data cube to represent the original data set, thus achieving data reduction.



4. Data Compression

Data compression employs modification, encoding, or converting the structure of data in a way that consumes less space. Data compression involves building a compact representation of information by removing redundancy and representing data in binary form. Data that can be restored successfully from its compressed form is called Lossless compression.



i. **Lossless Compression:** Encoding techniques (Run Length Encoding) allow a simple and minimal data size reduction. Lossless data compression uses algorithms to restore the precise original data from the compressed data.

يقلل حجم الملف عن طريق إزالة بعض البيانات الوصفية غير الضرورية

ii. **Lossy Compression:** In lossy-data compression, the decompressed data may differ from the original data but are useful enough to retrieve information from them. For example, the JPEG image format is a lossy compression, but we can find the meaning equivalent to the original image. Methods such as the Discrete Wavelet transform technique PCA (principal component analysis) are examples of this compression.

يقلل حجم الملف عن طريق إزالة بعض البيانات بشكل دائم

5. Discretization Operation

The **data discretization** technique is used to **divide the attributes of the continuous nature into data with intervals**. We replace many constant values of the attributes with labels of small intervals.

This means that mining results are shown in a concise and easily understandable way.

i. **Top-down discretization:** If you first consider one or a couple of points (so-called

breakpoints or split points) to divide the whole set of attributes and repeat this method up to the end, then the process is known as top-down discretization, also known as splitting.

ii. **Bottom-up discretization:** If you first consider all the constant values as split-points, some are discarded through a combination of the neighborhood values in the interval. That process is called bottom-up discretization.

Data Discretization

Data discretization refers to a method of converting a huge number of data values into smaller ones so that the evaluation and management of data become easy.

In other words, **data discretization** is a method of converting attributes values of continuous data into a finite set of intervals with minimum data loss.

There are **two** forms of data discretization: **supervised discretization** and **unsupervised discretization**.

Supervised discretization refers to a method in which the **class data is used**.

Unsupervised discretization refers to a method depending upon the way which operation proceeds. It means it works on the top-down splitting strategy and bottom-up merging strategy.

Now, we can understand this concept with the help of an example.

Example:

Suppose we have an attribute of **Age** with the given values.

Age :

1,5,9,4,7,11,14,17,13,18, 19,31,33,36,42,44,46,70,74,78,77

before discretization:			
Attribute	Age	Age	Age
	1,5,4,9,7	11,14,17,13,18,19	31,33,36,42,44,46
After Discretization	Child	Young	Mature

Some Famous techniques of data discretization

Histogram analysis

Histogram refers to a plot used to represent the underlying frequency distribution of a continuous data set. Histogram assists the data inspection for data distribution.

For example, Outliers, skewness representation, normal distribution representation, etc.

Binning

Binning refers to a data smoothing technique that helps to group a huge number of continuous values into smaller values. For data discretization and the development of idea hierarchy, this technique can also be used.

Cluster Analysis

Cluster analysis is a form of data discretization. A clustering algorithm is executed by dividing the values of x numbers into clusters to isolate a computational feature of x.

Data discretization using decision tree analysis

data discretization refers to a decision tree analysis in which a top-down slicing technique is used.

It is done through a supervised procedure. In a numeric attribute discretization, first, you need to select the attribute that has the least entropy, and then you need to run it with the help of a recursive process.

Data discretization and concept hierarchy generation

The term **hierarchy** represents an organizational structure or mapping in which items are ranked according to their levels of importance. In other words, we can say that a hierarchy concept refers to a sequence of mappings with a set of more general concepts to complex concepts. It means mapping is done from low-level concepts to high-level concepts.

Top-down mapping

Top-down mapping generally starts with the top with some general information and ends with the bottom to the specialized information.

Bottom-up mapping

Bottom-up mapping generally starts with the bottom with some specialized information and ends with the top to the generalized information.

Chapter Eleven – Statistics for Data Science

Descriptive statistics describe, show, and summarize the basic features of a dataset found in a given study, presented in a summary that describes the data sample and its measurements.

It helps analysts to understand the data better.

Descriptive statistics represent the available data sample and does not include theories, inferences, probabilities, or conclusions. That's a job for **inferential statistics**.

A good example of descriptive statistics:

look to a student's grade point average (GPA).

A GPA gathers the data points created through a large selection of grades, classes, and exams, then averages them together and presents a general idea of the student's mean academic performance.

Note that the GPA doesn't predict future performance or present any conclusions.

Types of Descriptive Statistics

Descriptive statistics break down into several types, characteristics, or measures. Some authors say that there are two types. Others say three or even four. In the spirit of working with averages, we will go with three types.

- **Distribution**, which deals with each value's frequency
- **Central tendency**, which covers the averages of the values

- **Variability** (or **dispersion**), which shows how spread out the values are **Distribution** (also called **Frequency Distribution**)

Datasets consist of a distribution of scores or values. Statisticians use **graphs** and **tables** to summarize the frequency of every possible value of a variable, rendered in percentages or numbers.

For instance, if you held a poll to determine people's favorite Beatle, you'd set up one column with all possible variables (John, Paul, George, and Ringo), and another with the number of votes.

Statisticians depict frequency distributions as either a **graph** or as a **table**.

Measures of Central Tendency

Measures of central tendency estimate a dataset's average or center, finding the result using three methods: **mean**, **mode**, and **median**.

Mean. The mean is also known as “M” and is the most common method for finding averages.

It is one of the first measurements we use to have a look at the data is to obtain sample statistics from the data, such as the sample mean. Given a sample of n values, $\{x_i\}, i = 1, \dots, n$, the mean, μ , is the sum of the values divided by the number of values,

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i.$$

For instance, say someone is trying to figure out how many hours a day they sleep in a week. So, the data set would be the hour entries (e.g., 6,8,7,10,8,4,9), and the sum of those values is 52. There are seven responses, so $N=7$. You divide the value sum of 52 by N , or 7, to find M , which in this instance is 7.3.

```
import pandas as pd

# list of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]

# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}
df = pd.DataFrame(dict)

print(df)

print()

print()

fm['score'].mean()
```

```
In [56]: import pandas as pd
# List of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]

# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}

df = pd.DataFrame(dict)
print(df)
print()
print()
fm['score'].mean()
```

	name	degree	score
0	aparna	MBA	90
1	pankaj	BCA	40
2	sudhir	M.Tech	80
3	Geeku	MBA	98

Out[56]: 40.0

Mode. The mode is just the most frequent response value. Datasets may have any number of modes, including “zero.” You can find the mode by arranging your dataset's order from the lowest to highest value and then looking for the most common response. So, in using our sleep study from the last part: 4,6,7,8,8,9,10. As you can see, the mode is eight.

Median. Finally, we have the median, defined as the value in the precise center of the dataset.

Arrange the values in ascending order (like we did for the mode) and look for the number in the set’s middle. In this case, the median is **eight**.

Sample median

The **mean** of the samples is a good descriptor, but it has an important drawback: what will happen if in the sample set there is an error with a value very different from the rest?

For example, considering hours worked per week, it would normally be in a range between 20 and 80; but what would happen if by mistake there was a value of 1000? An

item of data that is significantly different from the rest of the data is called an **outlier**. In this case, the mean, μ , will be drastically changed towards the **outlier**. One solution to this drawback is offered by the statistical median, μ_{12} , which is an order statistic giving the middle value of a sample

```
print(fm['score'].median())
```

Variability (also called Dispersion)

The measure of variability gives the statistician an idea of how spread out the responses are. The spread has three aspects — **range**, **standard deviation**, and **variance**.

Range. Use range to determine how far apart the most extreme values are. Start by subtracting the dataset's lowest value from its highest value. Once again, we turn to our sleep study:

4,6,7,8,8,9,10. We subtract four (the lowest) from ten (the highest) and get six. There's your range.

Standard Deviation. This aspect takes a little more work. The standard deviation (s) is your dataset's average amount of variability, showing you how far each score lies from the mean. The larger your standard deviation, the greater your dataset's variable. Follow these six steps:

1. List the scores and their means.
2. Find the deviation by subtracting the mean from each score.
3. Square each deviation.
4. Total up all the squared deviations.
5. Divide the sum of the squared deviations by $N-1$.

6. Find the result's square root.

Example: we turn to our sleep study: 4,6,7,8,8,9,10.

Raw number/data	Deviation from mean	Deviation Squared
4	$4-7.3 = -3.3$	10.89
6	$6-7.3 = -1.3$	1.69
7	$7-7.3 = -0.3$	0.09
8	$8-7.3 = 0.7$	0.49
8	$8-7.3 = 0.7$	0.49
9	$9-7.3 = 1.7$	2.89
10	$10-7.3 = 2.7$	7.29
M=7.3	Sum = 0.9	Square sums= 23.83

When you divide the sum of the squared deviations by 6 (N-1): $23.83/6$, you get 3.971, and the square root of that result is 1.992.

As a result, we now know that each score deviates from the mean by an average of 1.992 points.

Variance: Variance reflects the dataset's degree spread. The greater the degree of data spread, the larger the variance relative to the mean. You can get the variance by just squaring the standard deviation. Using our above example, we square 1.992 and arrive at 3.971.

Sample variance

The mean is not usually a sufficient descriptor of the data. We can go further by knowing two numbers: mean and variance.

$$\sigma^2 = \frac{1}{n} \sum_i (x_i - \mu)^2.$$

```
print(fm['score'].var())
```

Measuring Asymmetry: Skewness and Pearson's Median Skewness Coefficient

Skewness:

Skewness is the measure of the **asymmetry** of an ideally symmetric probability distribution and is given by the third standardized moment.

In simple words, skewness is the measure of how much the probability distribution of a random variable deviates from the normal distribution.

For univariate **أحادي المتغير** data, the formula for **skewness** is a statistic that measures the asymmetry of the set of n data samples, x_i :

$$g_1 = \frac{1}{n} \frac{\sum_i (x_i - \mu)^3}{\sigma^3},$$

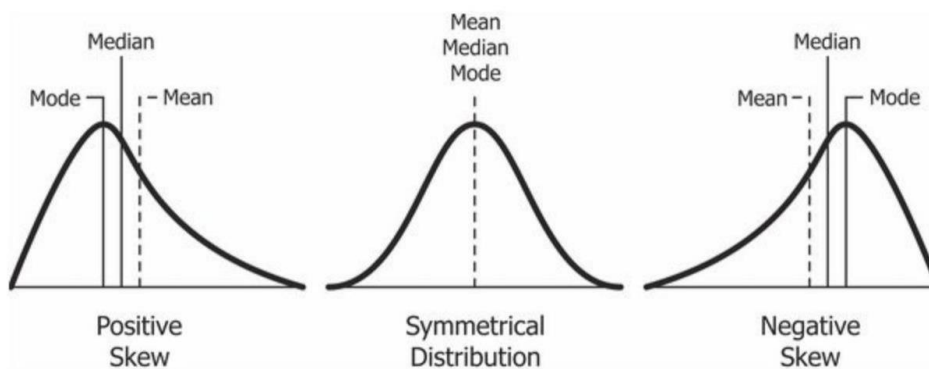
where μ is the mean, σ is the standard deviation, and n is the number of data points.

Negative deviation indicates that the distribution “**skews left**” (it extends further to the left than to the right).

Note: the skewness for a **normal distribution** is **zero**, and any **symmetric** data must have a skewness of zero.

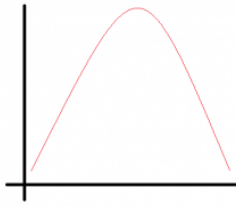
The **normal distribution** is the probability distribution **without any skewness**. You can look at the image below which shows symmetrical distribution that's basically a normal distribution and you can see that it is symmetrical on both sides of the dashed line. Apart from this, **there are three types of skewness**:

- Positive Skewness
- Negative Skewness
- Zero skewness



The probability distribution with its tail on the right side is a positively skewed distribution and the one with its tail on the left side is a negatively skewed distribution.

- **Normal Distribution:** Normal Distribution is a probability distribution that is symmetric about the mean. It is also known as Gaussian Distribution. The distribution appears as a Bell-shaped curve which means the mean is the most frequent data in the given data set.



In Normal Distribution :

Mean = Median = Mode

- **Standard Normal Distribution:** When in the Normal Distribution **mean = 0** and the Standard Deviation = 1 then Normal Distribution is called as Standard Normal Distribution.
- Normal Distributions are symmetrical in nature it doesn't imply that every symmetrical distribution is a Normal Distribution.
- Normal Distribution is the probability distribution without any skewness.

Unlike the Normal Distribution (mean = median = mode), in positive as well as negative skewness mean, median, and mode all are different.

Positive Skewness

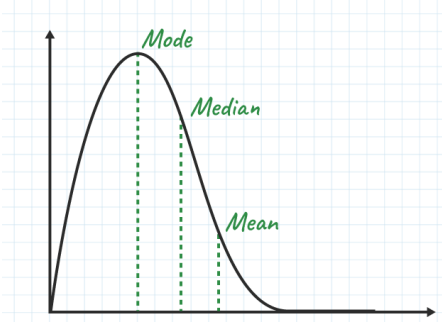
In positive skewness, the extreme data values are larger, which in turn increase the mean value of the data set, or in the simple term in positive skew distribution is the distribution having the tail on the right side.

Positive Skewness means the **tail** on the **right** side of the distribution is **longer**.

The mean and median will be greater than the mode.

Condition for positive skewness = **Mean > Median > Mode**

Positive Skew



Let's take an example of the income distribution where a **few people earn very high incomes and the majority earn lower incomes**. so, this is often **positively** skewed. Analyzing skewed data can provide valuable insights into the underlying causes and potential solutions or interventions.

In Positive Skewness:

Mean > Median > Mode

Negative Skewness

In negative skewness, the extreme data values are smaller, which decreases the mean value of the dataset or the negative skew distribution is the distribution having the tail on the left side.

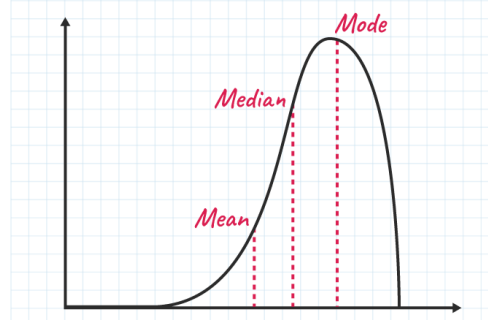
Negative Skewness means when the **tail** of the **left** side of the distribution is **longer** than the tail on the right side.

The **mean** and **median** will be **less** than the **mode**.

Condition for negative skewness is **Mode > Median > Mean**

The curve shows negative skewness in the image below,

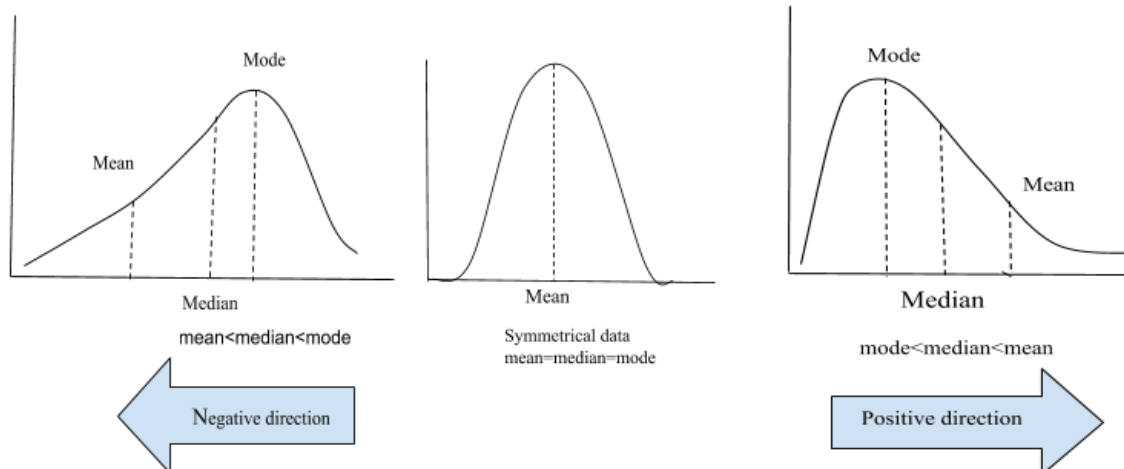
Negative Skew



Let's take an example of a match, during the match most of the players of a particular team scored runs above 50 and only a few of them scored below 10. In such a case, the data is generally represented with the help of a negatively skewed distribution. And this data is helpful to analyze the game's performance.

In Negative Skewness:

Mean < Median < Mode



Zero Skewness

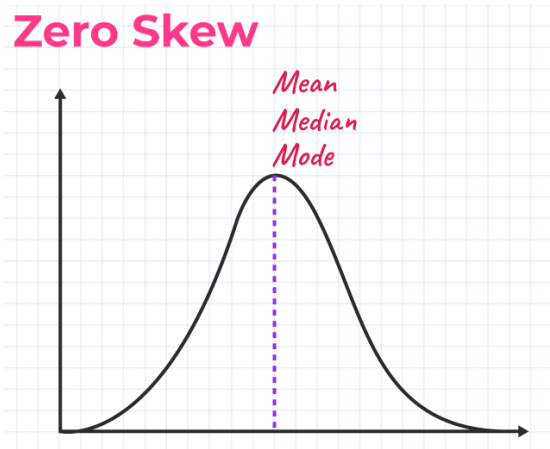
It is also known as a “symmetric distribution”.

It signifies that distribution of data is **evenly distributed around the mean**, with no long tails on either end of the distribution.

Condition for zero skewness is **Mean = Mode = Median**

The curve for zero skewness is shown in the image below,

Zero Skew



Calculate the skewness coefficient of the sample

Pearson's first coefficient of skewness

Subtract a mode from a mean, then divides the difference by standard deviation.

$$\text{Pearson's first coefficient} = \frac{\text{Mean} - \text{Mode}}{\text{Standard Deviation}}$$

As **Pearson's correlation coefficient** differs from **-1** (perfect negative linear relationship) to **+1** (perfect positive linear relationship), including a value of 0 indicating no linear relationship.

When we divide the covariance values by the standard deviation, it truly scales the value down to a limited range of **-1 to +1**. That accurately the range of the correlation values.

Pearson's second coefficient of skewness

Multiply the difference by 3, and divide the product by standard deviation.

$$\text{Pearson's second coefficient} = \frac{3 (\text{Mean} - \text{Median})}{\text{Standard Deviation}}$$

$$\text{Mean} - \text{Mode} \approx 3 (\text{Mean} - \text{Median})$$

If the **skewness** is between -0.5 & 0.5, the data are **nearly symmetrical**.

If the skewness is between -1 & -0.5 (negative skewed) or between 0.5 & 1 (positive skewed), the data are **slightly skewed**.

If the skewness is lower than -1 (**negative skewed**) or greater than 1 (**positive skewed**), the data are **extremely skewed**.

The **Pearson's median skewness coefficient** is a more robust alternative to the skewness coefficient and is defined as follows:

$$g_p = 3(\mu - \mu_{12})\sigma$$

μ mean, μ_{12} median

def **pearson**(x):

 return 3*(x.mean() - x.median())*x.std()

print "Pearson's coefficient of x = ", pearson(x)

Kurtosis:

Kurtosis measures the "heaviness of the tails" of a distribution (in compared to a normal

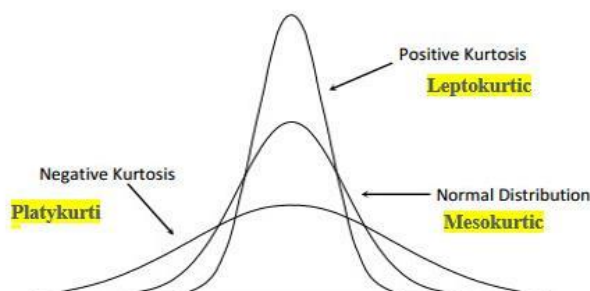
distribution). Kurtosis is positive if the tails are "heavier" than for a normal distribution, and negative if the tails are "lighter" than for a normal distribution. The normal distribution has kurtosis of zero.

Kurtosis characterizes the shape of a distribution - that is, its value does not depend on an arbitrary change of the scale and location of the distribution. For example, kurtosis of a sample (or population) of temperature values in Fahrenheit will not change if you transform the values to Celsius (the mean and the variance will, however, change).

The kurtosis of a distribution or sample is equal to the 4th central moment divided by the 4th power of the standard deviation, minus 3.

To calculate the kurtosis of a sample:

- i) subtract the mean from each value to get a set of deviations from the mean;
- ii) divide each deviation by the standard deviation of all the deviations;
- iii) average the 4th power of the deviations and subtract 3 from the result.



Excess Kurtosis

The excess kurtosis is used in statistics and probability theory to compare the kurtosis coefficient with that normal distribution. Excess kurtosis can be positive (Leptokurtic

distribution), negative (Platykurtic distribution), or near to zero (Mesokurtic distribution).

Since normal distributions have a kurtosis of 3, excess kurtosis is calculated by subtracting kurtosis by 3.

$$\text{Excess kurtosis} = \text{Kurt} - 3$$

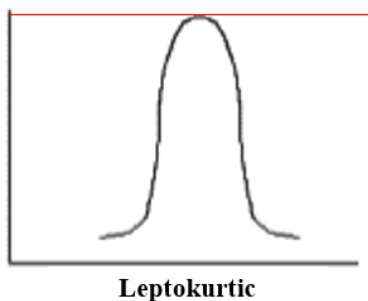
Types of excess kurtosis

1. *Leptokurtic or heavy-tailed distribution* (kurtosis more than normal distribution).
2. *Mesokurtic* (kurtosis same as the normal distribution).
3. *Platykurtic or short-tailed distribution* (kurtosis less than normal distribution).

Leptokurtic (kurtosis > 3)

Leptokurtic is having very long and skinny tails, which means there are more chances of outliers.

Positive values of kurtosis indicate that distribution is peaked and possesses thick tails. An extreme positive kurtosis indicates a distribution where more of the numbers are located in the tails of the distribution instead of around the mean.

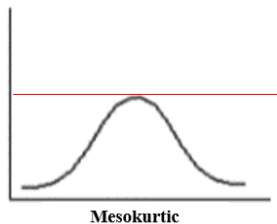


platykurtic (kurtosis < 3)

Platykurtic having a lower tail and stretched around center tails means most of the data points are present in high proximity with mean. A platykurtic distribution is flatter (less peaked) when compared with the normal distribution.

Mesokurtic (kurtosis = 3)

Mesokurtic is the same as the normal distribution, which means kurtosis is near to 0. In Mesokurtic, distributions are moderate in breadth, and curves are a medium peaked height.



$$\text{Mesokurtic} = 3 - 3 = 0$$

Covariance, and Pearson's and Spearman's Rank Correlation

Variables of data can express relations. For example, countries that tend to invest in research also tend to invest more in education and health. This kind of relationship is captured by the covariance.

يمكن لمتغيرات البيانات التعبير عن العلاقات. على سبيل المثال، تميل البلدان التي تميل إلى الاستثمار في البحوث إلى زيادة الاستثمار في التعليم والصحة. يتم التقاط هذا النوع من العلاقات من خلال التغيرات.

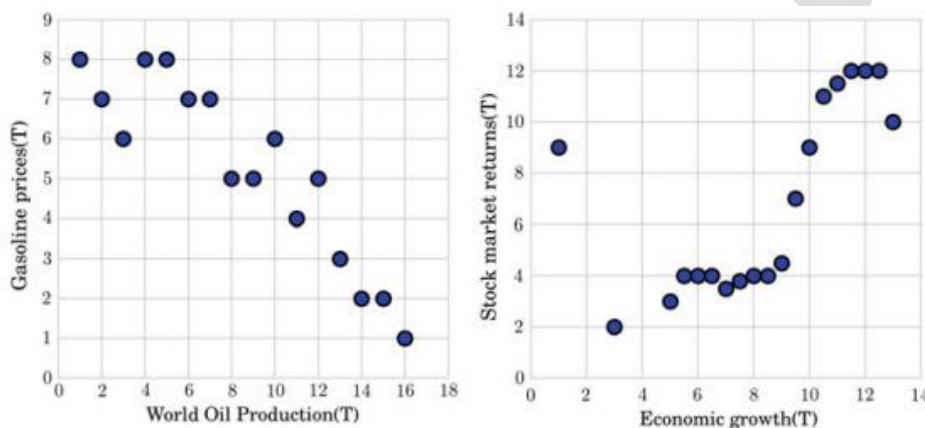


Fig. 3.9 Positive correlation between economic growth and stock market returns worldwide (*left*). Negative correlation between the world oil production and gasoline prices worldwide (*right*)

Covariance is a measure of the relationship between two random variables and to what extent, they change together. Or we can say, in other words, it defines the changes between the two variables, such that change in one variable is equal to change in another variable. This is the property of a function of maintaining its form when the variables are linearly transformed. Covariance is measured in units, which are calculated by multiplying the units of the two variables.

Population Covariance Formula

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N}$$

Sample Covariance

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N - 1}$$

Types of Covariance

Covariance can have both **positive** and **negative** values. Based on this, it has two types:

- 1- Positive Covariance
- 2- Negative Covariance

Positive Covariance

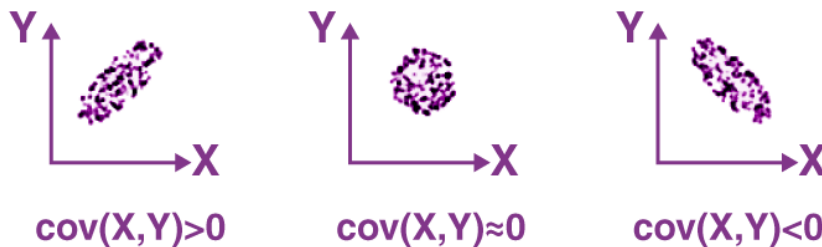
If the covariance for any two variables is positive, that means, both the variables move in the same direction.

Here, the variables show similar behavior.

That means, if the values (greater or lesser) of one variable corresponds to the values of another variable, then they are said to be in positive covariance.

Negative Covariance

If the covariance for any two variables is negative, that means, both the variables move in the opposite direction. It is the opposite case of positive covariance, where greater values of one variable correspond to lesser values of another variable and vice-versa.



If $\text{cov}(X, Y)$ is greater than zero, then we can say that the covariance for any two variables is positive and both the variables move in the same direction.

If $\text{cov}(X, Y)$ is less than zero, then we can say that the covariance for any two variables is negative and both the variables move in the opposite direction.

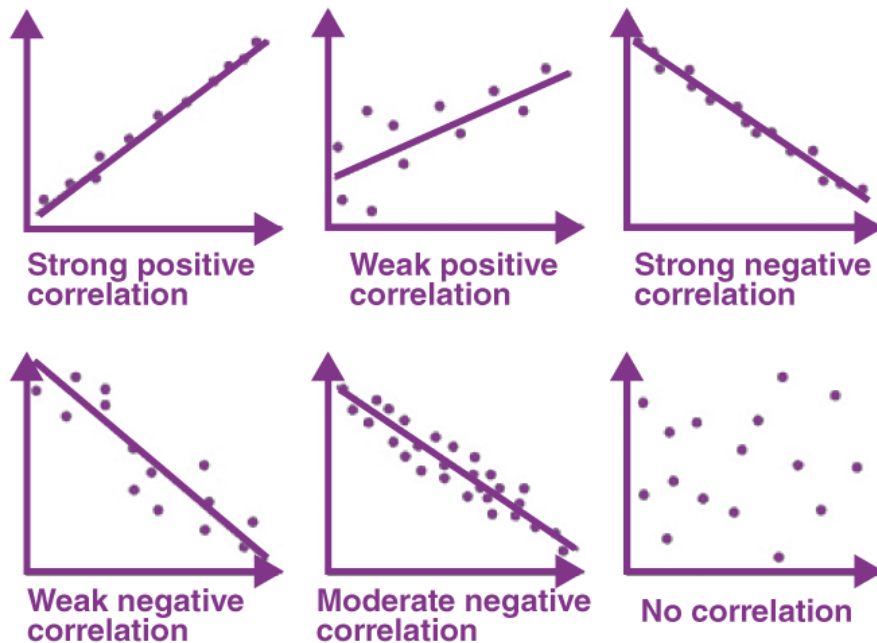
If $\text{cov}(X, Y)$ is zero, then we can say that there is no relation between two variables.

Correlation Coefficient Formula

We have already discussed covariance, which is the evaluation of changes between any two variables. Correlation estimates the depth of the relationship between variables. It is the estimated measure of covariance and is dimensionless. In other words, the correlation coefficient is a constant value always and does not have any units. The relationship between the correlation coefficient and covariance is given by;

$$\text{Correlation}, \rho(X, Y) = \text{Cov}(X, Y) / \sigma_X \sigma_Y$$

Based on the value of correlation coefficient, we can estimate the type of correlation between the given two variables. Also, the graphical representation of correlation among two variables is given in the below figure.



Note that the **Pearson's correlation** is always between -1 and $+1$, where the magnitude depends on the degree of correlation.

If the Pearson's correlation is 1 (or -1), it means that the variables are **perfectly** correlated (positively or negatively)

Spearman's Rank Correlation

The Spearman's rank correlation comes as a solution to the robustness problem of Pearson's correlation when the data contain outliers.

The main idea is to use the **ranks** of the **sorted** sample data, instead of the values themselves.

For example, in the list $[4, 3, 7, 5]$, the rank of 4 is 2 , since it will appear second in the ordered list $([3, 4, 5, 7])$.

Spearman's correlation computes the correlation between the ranks of the data.

For example, considering the data: $X=[10, 20, 30, 40, 1000]$, and $Y=[-70, -1000, -50, -10, -20]$, where we have an outlier in each one set.

If we compute the ranks, they are $[1.0, 2.0, 3.0, 4.0, 5.0]$ and $[2.0, 1.0, 3.0, 5.0, 4.0]$.

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

As value of the Pearson's coefficient, we get 0.28, which does not show much correlation between the sets. However, the Spearman's rank coefficient, capturing the correlation between the ranks, gives as a final value of 0.80, confirming the correlation between the sets. As an exercise, you can compute the Pearson's and the Spearman's rank correlations for the different Anscombe configurations given in Fig. 3.10. Observe if linear and nonlinear correlations can be captured by the Pearson's and the Spearman's rank correlations.

Example:

consider the following example data regarding the marks achieved in a maths and English exam:

	Marks									
	5	7	4	7	6	6	5	8	7	6
English	6	5	5	1	1	4	8	0	6	1
Maths	6	7	4	6	6	5	5	7	6	6
	6	0	0	0	5	6	9	7	7	3

The procedure for ranking these scores is as follows:

First, create a table with four columns and label them as below:

English (mark)	Maths (mark)	Rank (English)	Rank (maths)
56	66	9	4
75	70	3	2
45	40	10	10
71	60	4	7

English (mark)	Maths (mark)	Rank (English)	Rank (maths)
61	65	6.5	5
64	56	5	9
58	59	8	8
80	77	1	1
76	67	2	3
61	63	6.5	6

You need to rank the scores for maths and English separately. The score with the highest value should be labelled "1" and the lowest score should be labelled "10" (if your data set has more than 10 cases then the lowest score will be how many cases you have). Look carefully at the two individuals that scored 61 in the English exam (highlighted in bold). Notice their joint rank of 6.5. This is because when you have two identical values in the data (called a "tie"), you need to take the average of the ranks that they would have otherwise occupied. We do this because, in this example, we have no way of knowing which score should be put in rank 6 and which score should be ranked 7. Therefore, you will notice that the ranks of 6 and 7 do not exist for English. These two ranks have been averaged ($(6 + 7)/2 = 6.5$) and assigned to each of these "tied" scores.

An example of calculating Spearman's correlation

To calculate a Spearman rank-order correlation on data without any ties we will use the following data:

	Marks									
English	56	75	45	71	62	64	58	80	76	61
Maths	66	70	40	60	65	56	59	77	67	63

We then complete the following table:

English (mark)	Maths (mark)	Rank (English)	Rank (maths)	d	d ²
56	66	9	4	5	25
75	70	3	2	1	1
45	40	10	10	0	0
71	60	4	7	3	9
62	65	6	5	1	1
64	56	5	9	4	16
58	59	8	8	0	0
80	77	1	1	0	0
76	67	2	3	1	1
61	63	7	6	1	1

Where d = difference between ranks and d² = difference squared.

We then calculate the following:

$$\sum d_i^2 = 25 + 1 + 9 + 1 + 16 + 1 + 1 = 54$$

We then substitute this into the main equation with the other information as follows:

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

$$\rho = 1 - \frac{6 \times 54}{10(10^2 - 1)}$$

$$\rho = 1 - \frac{324}{990}$$

$$\rho = 1 - 0.33$$

$$\rho = 0.67$$

as $n = 10$. Hence, we have a ρ (or r_s) of 0.67. This indicates a strong positive relationship between the ranks individuals obtained in the maths and English exam.

That is, the higher you ranked in maths, the higher you ranked in English also, and vice versa.

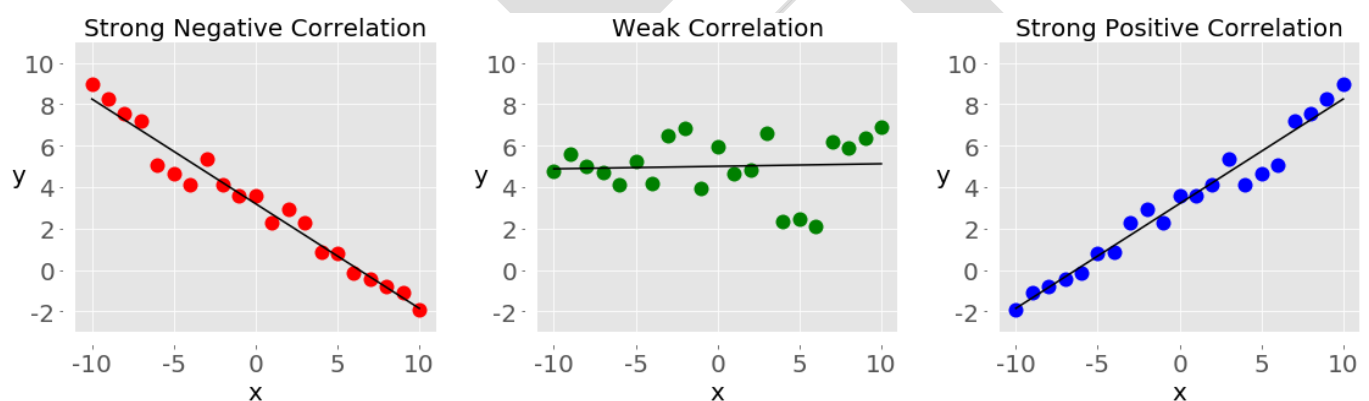
Example:

When data is represented in the form of a table, the rows of that table are usually the observations, while the columns are the features. Take a look at this employee table:

Name	Years of Experience	Annual Salary
Ann	30	120,000
Rob	21	105,000
Tom	19	90,000
Ivy	10	82,000

In this table, each row represents one observation, or the data about one employee (either Ann, Rob, Tom, or Ivy). Each column shows one property or feature (name, experience, or salary) for all the employees.

If you analyze any two features of a dataset, then you'll find some type of **correlation** between those two features. Consider the following figures:

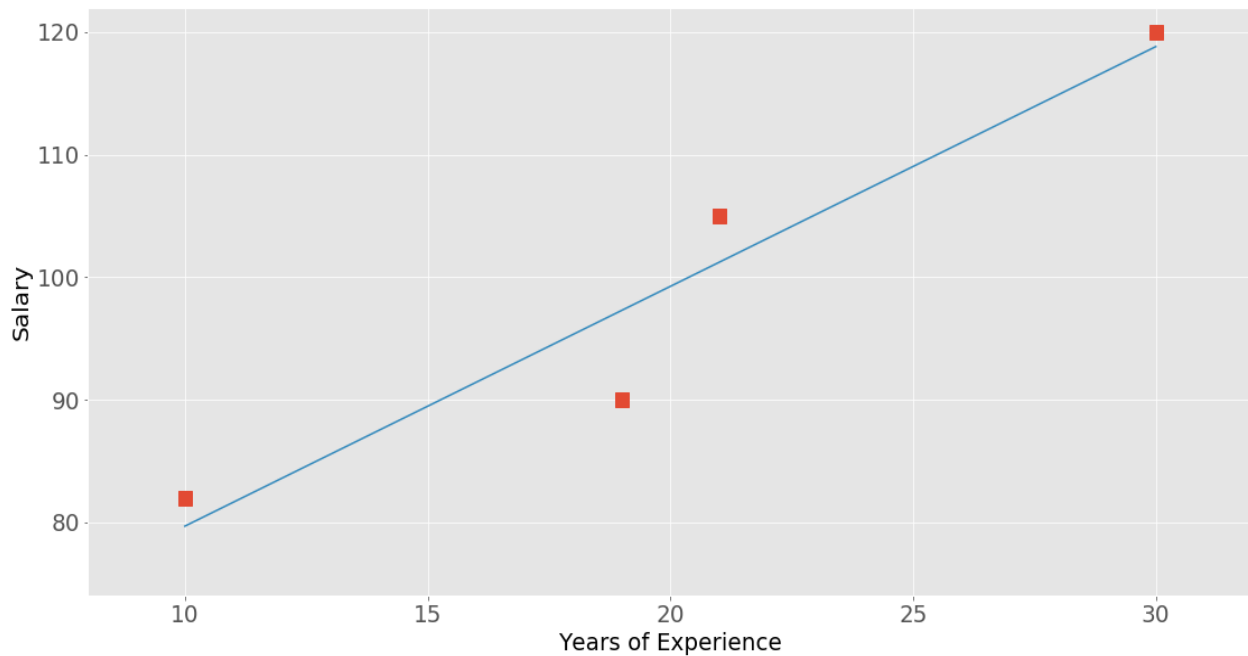


Each of these plots shows one of three different forms of correlation:

1. **Negative correlation (red dots):** In the plot on the left, the y values tend to decrease as the x values increase. This shows strong negative correlation, which occurs when *large* values of one feature correspond to *small* values of the other, and vice versa.
2. **Weak or no correlation (green dots):** The plot in the middle shows no obvious trend. This is a form of weak correlation, which occurs when an association between two features is not obvious or is hardly observable.

3. **Positive correlation (blue dots):** In the plot on the right, the y values tend to increase as the x values increase. This illustrates strong positive correlation, which occurs when *large* values of one feature correspond to *large* values of the other, and vice versa.

The next figure represents the data from the employee table above:



The correlation between experience and salary is positive because higher experience corresponds to a larger salary and vice versa.

```
In [21]: import numpy as np
x = np.arange(10, 20)
print("x=",x)
# array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
y = np.array([2, 1, 4, 5, 8, 12, 18, 25, 96, 48])
print("y=",y)
# array([ 2,  1,  4,  5,  8, 12, 18, 25, 96, 48])
r = np.corrcoef(x, y) # Pearson correlation coefficient
print("r=",r)
print("r[0, 1]",r[0, 1])
print("r[1, 0]",r[1, 0])
```

```
x= [10 11 12 13 14 15 16 17 18 19]
y= [ 2  1  4  5  8 12 18 25 96 48]
r= [[1.          0.75864029]
     [0.75864029 1.          ]]
r[0, 1] 0.7586402890911867
r[1, 0] 0.7586402890911866
```

```
In [ ]:
```

The values on the main diagonal of the correlation matrix (upper left and lower right) are equal to 1. The upper left value corresponds to the correlation coefficient for x and x , while the lower right value is the correlation coefficient for y and y . They are always equal to 1.

Kendal tau Rank Correlation

The Kendall tau-b correlation coefficient, τ_b , is a nonparametric measure of association based on the number of concordances and discordances in paired observations.

Suppose two observations (X_i, Y_i) and (X_j, Y_j) are **concordant** if they are in the same order with respect to each variable. That is, if

1. $X_i < X_j$ and $Y_i < Y_j$, or if
2. $X_i > X_j$ and $Y_i > Y_j$

They are **discordant** if they are in the reverse ordering for X and Y , or the values are arranged in opposite directions. That is, if

1. $X_i < X_j$ and $Y_i > Y_j$, or if
2. $X_i > X_j$ and $Y_i < Y_j$

The two observations are **tied** if $X_i = X_j$ and/or $Y_i = Y_j$

The total number of pairs that can be constructed for a sample size of n is

$$N = \binom{n}{2} = \frac{1}{2}n(n-1)$$

N can be decomposed into these five quantities:

$$N = P + Q + X_0 + Y_0 + (XY)_0$$

where P is the number of concordant pairs, Q is the number of discordant pairs, X_0 is the number of pairs tied only on the X variable, Y_0 is the number of pairs tied only on the Y variable, and $(XY)_0$ is the number of pairs tied on both X and Y .

The Kendall tau-b for measuring order association between variables X and Y is given by the following formula:

$$t_b = \frac{P - Q}{\sqrt{(P + Q + X_0)(P + Q + Y_0)}}$$

-1:1

Example:

Age and **percentage** body fat were measured in 18 adults. Calculate each of the Pearson, Spearman, and Kendall correlation coefficients. AS:

No. Age fat%

01	23	9.5
02	23	27.9
03	27	7.8
04	27	17.8
05	39	31.4
06	41	25.9
07	45	27.4
08	49	25.2
09	50	31.1
10	53	34.7

11	53	42.0
12	54	29.1
13	56	32.5
14	57	30.3
15	58	33.0
16	58	33.8
17	60	41.1
18	61	34.5

Question:

Calculate the coefficient of covariance for the following data:

X	2	8	18	20	28	30
Y	5	12	18	23	45	50

Solution:

Number of observations = 6

Mean of X = 17.67

Mean of Y = 25.5

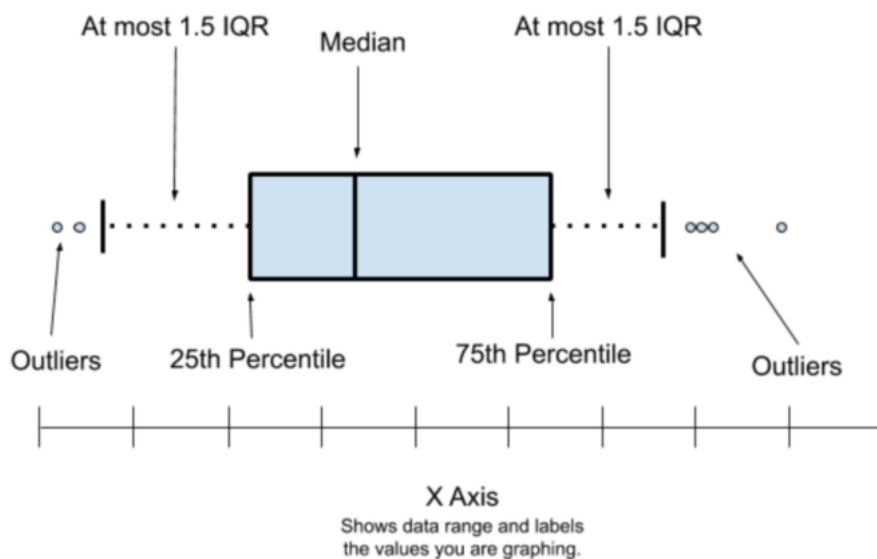
$$\begin{aligned} \text{Cov}(X, Y) &= \left(\frac{1}{n}\right) [(2 - 17.67)(5 - 25.5) + (8 - 17.67)(12 - 25.5) + (18 - 17.67)(18 - 25.5) + \\ &\quad (20 - 17.67)(23 - 25.5) + (28 - 17.67)(45 - 25.5) + (30 - 17.67)(50 - 25.5)] \\ &= 157.83 \end{aligned}$$

Box plot:

A box plot also known as [Five Number Summary](#), summarizes data using the median, upper quartile, lower quartile, and the minimum and maximum values. It allows you to see important characteristics of the data at a glance(visually). This also help us to visualize outliers in the data set.

Box plot or Five Number Summary has below five information.

1. Median
2. Lower Quartile (25th Percentile)
3. Upper Quartile (75th Percentile)
4. Minimum Value
5. Maximum Value



Example of Box

Step 1 — take the set of numbers given

14, 19, 100, 27, 54, 52, 93, 50, 61, 87, 68, 85, 75, 82, 95

Arrange the data in increasing(ascending) order

14, 19, 27, 50, 52, 54, 61, 68, 75, 82, 85, 87, 93, 95, 100

Step 2 — Find the median of this data set. Median is mid value in this ordered data set.

14, 19, 27, 50, 52, 54, 61, **68**, 75, 82, 85, 87, 93, 95, 100

Here it is 68.

Step 3 — Lets find the Lower Quartile.

Lower Quartile is the median from the left of the, medium found in the Step 2(ie. 68)

(14, 19, 27, **50**, 52, 54, 61), 68, 75, 82, 85, 87, 93, 95, 100

Lower Quartile is 50

Step 4 — Lets find the Upper Quartile.

Upper Quartile is the median from the Right of the medium found in the Step 2(ie. 68)

14, 19, 27, 50, 52, 54, 61, 68,(75, 82, 85, **87**, 93, 95, 100)

Upper Quartile is 87

Step 5 — Lets find the Minimum Value

It is value lies in the extreme left from this data set or first value in the data set after ordering.

14, 19, 27, 50, 52, 54, 61, 68, 75, 82, 85, 87, 93, 95, 100

Minimum Value is 14

Step 6 — Lets find the Maximum Value

It is value lies in the extreme Right from this data set or last value in the data set after ordering.

14, 19, 27, 50, 52, 54, 61, 68, 75, 82, 85, 87, 93, 95, **100**

Maximum Value is 100

Median	=> 68
Lower Quartile	=> 50
Upper Quartile	=> 87
Minimum Value	=> 14
Maximum Value	=> 100

Range :

Range is basically spread of our data set. Range can be found as difference between Maximum Value and Minimum Values.

$$\text{Range} = \text{Maximum Value} - \text{Minimum Value}$$
$$\text{Range} = 100 - 14 = 86$$

Quantiles and Percentiles

we can order the samples $\{x_i\}$, then find the x_p so that it divides the data into two parts, where

- a fraction p of the data values is less than or equal to x_p and
- the remaining fraction $(1 - p)$ is greater than x_p .

That value, x_p , is the p -th quantile, or the $100 \times p$ -th percentile.

For example, a 5-number summary is defined by the values $x_{\min}$, $Q1$, $Q2$, $Q3$, $x_{\max}$, where $Q1$ is the $25 \times p$ -th percentile, $Q2$ is the $50 \times p$ -th percentile and $Q3$ is the $75 \times p$ -th percentile.

Interquartile Range(IQR):

Interquartile Range(IQR) is difference between Upper quartile and Lower quartile.

As per picture above, our Lower quartile is 50 and Upper quartile is 87

$IQR = \text{Upper Quartile} - \text{Lower Quartile}$

$IQR = 87 - 50 = 27$

Box plot with Even numbers of data set :

Step 1 :

We have 14 records below.

14, 19, 100, 27, 54, 52, 93, 50, 61, 87, 68, 85, 75, 82

Arrange the data in increasing(ascending) order

14, 19, 27, 50, 52, 54, 61, 68, 75, 82, 85, 87, 93, 100

Step 2: Since we have the even number take the middle two values add them and divide them by 2.

Here Values at position 7 & 8 are middle values.

							64.5							
							↑							
Numbers =>	14,	19,	27,	50,	52,	54,	61,	68,	75,	82,	85,	87,	93,	100
Position =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14

So our new median value is 64.5

Numbers =>	14,	19,	27,	50,	52,	54,	61,	64.5,	68,	75,	82,	85,	87,	93,	100
Position =>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Continue Step 3 to Step 6 to get the values mention in **Working Example of Box**

Plot section. Final Result as below

Median	=> 64.5
Lower Quartile	=> 50
Upper Quartile	=> 85
Minimum Value	=> 14
Maximum Value	=> 100

Pivot table:

A pivot table is a **summary tool** that wraps up or summarizes information sourced from bigger tables. These bigger tables could be a database, an Excel spreadsheet, or any data that is or could be converted in a table-like form. The data summarized in a pivot table might include sums, averages, or other statistics which the pivot table groups together in a meaningful way.

Wide vs. Long Data

Pivoting is the act of converting “long data” into “wide data”. Wide and long data formats serve different purposes. It’s often helpful to think of how data might be collected in the first place.

Representing potentially multi-dimensional data in a single 2-dimensional table may require some compromises. Either some data will be repeated (long data format) or your data set may require blank cells (wide data).

Examples of pivot table:

- **Pivoting in Python with Pandas**

Starting with the raw dataset loaded into `df_long_data`

```
df_wide_data = df_long_data.pivot_table(  
    index = ["GEO", ],  
    columns = ["Vehicle type", "Year"],  
    values = ["VALUE"],  
    aggfunc = "sum"  
)
```

1. **Index** determines what will be unique in the leftmost column of the result.
2. **Columns** creates a column for each unique value in the “Vehicle type” and “Year” columns of the input table.
3. **Values** defines what to put in the cells of the output table.
4. **aggfunc** defines *how* to combine the values (commonly sum, but other aggregation functions are also used: min, max, mean, 95th percentile, mode). You can also define your own aggregation functions and pass in the function.

• Pivoting in Google Sheets

Pivoting in Google sheets is doing the exact same thing as our python code above but in a graphical way. Instead of specifying arguments for the pivot_table function, we select from dropdowns. But the important thing to remember is pivoting doesn’t “belong” to any particular software, it’s a generalized approach for dealing with data.

• Grouping in PostgreSQL

Many relational databases don’t have a built in pivot function. We can write a query that approximates the desired results but it does require some manual intervention to define the possible groups.

Heat map:

Heatmaps visualize the data in a 2-dimensional format in the form of colored maps. The color maps use hue, saturation, or luminance to achieve color variation to display various details. This color variation gives visual cues to the readers about the magnitude of numeric values.

Heatmaps can describe the density or intensity of variables, visualize patterns, variance, and even anomalies. Heatmaps show relationships between variables. These variables are plotted on both axes.

Uses of HeatMap

Business Analytics: A heat map is used as a visual business analytics tool. A heat map gives quick visual cues about the current results, performance, and scope for improvements. Heatmaps can analyze the existing data and find areas of intensity that might reflect where most customers reside, areas of risk of market saturation, or cold sites and sites that need a boost. Heat maps can be continued to be updated to reflect the growth and efforts. These maps can be integrated into a business's workflow and become a part of ongoing analytics.

- **Website:** Heatmaps are used in websites to visualize data of visitors' behavior. This visualization helps business owners and marketers to identify the best & worst-performing sections of a webpage. These insights help them with optimization.

- **Exploratory Data Analysis:** EDA is a task performed by data scientists to get familiar with the data. All the initial studies are done to understand the data are known as EDA. Exploratory Data Analysis (EDA) is the process of analyzing datasets before the modeling task. It is a tedious task to look at a spreadsheet filled with numbers and determine

essential characteristics in a dataset

- **Molecular Biology:** Heat maps are used to study disparity and similarity patterns in DNA, RNA, etc.
- **Geovisualization:** Geospatial heatmap charts are useful for displaying how geographical areas of a map are compared to one another based on specific criteria. Heatmaps help in cluster analysis or hotspot analysis to detect clusters of high concentrations of activity; For example, Airbnb rental price analysis.
- **Marketing and Sales:** The heatmap's capability to detect warm and cold spots is used to improve marketing response rates by targeted marketing. Heatmaps allow the detection of areas that respond to campaigns, under-served markets, customer residence, and high sale trends, which helps optimize product lineups, capitalize on sales, create targeted customer segments, and assess regional demographics.

Types of HeatMaps

Typically, there are two types of Heatmaps:

- **Grid Heatmap:** The magnitudes of values shown through colors are laid out into a matrix of rows and columns, mostly by a density-based function. Below are the types of Grid Heatmaps.
- ✓ **Clustered Heatmap:** The goal of Clustered Heatmap is to build associations between both the data points and their features. This type of heatmap implements clustering as part of the process of grouping similar features. Clustered Heatmaps are widely used in biological sciences for studying gene similarities across individuals.

The order of the rows in Clustered Heatmap is determined by performing hierarchical cluster analysis of the rows. Clustering positions similar rows together on the map. Similarly, the order of the columns is determined.

✓ **Correlogram:** A correlogram replaces each of the variables on the two axes with numeric variables in the dataset. Each square depicts the relationship between the two intersecting variables, which helps to build descriptive or predictive statistical models.

- **Spatial Heatmap:** Each square in a Heatmap is assigned a color representation according to the nearby cells' value. The location of color is according to the magnitude of the value in that particular space. These Heatmaps are data-driven “paint by numbers” canvas overlaid on top of an image. The cells with higher values than other cells are given a hot color, while cells with lower values are assigned a cold color.

Correlation statistics:

Correlation

➤ Correlation measures the relationship between two variables.

We mentioned that a function has a purpose to predict a value, by converting input (x) to output ($f(x)$). We can also say that a function uses the relationship between two variables for prediction.

Correlation Coefficient

The correlation coefficient measures the relationship between two variables.

The correlation coefficient can never be less than -1 or higher than 1.

- 1 = there is a perfect linear relationship between the variables (like Average_Pulse against Calorie_Burnage)
- 0 = there is no linear relationship between the variables
- -1 = there is a perfect negative linear relationship between the variables (e.g. Less hours worked, leads to higher calorie burnage during a training session)

Example of a Perfect Linear Relationship (Correlation Coefficient = 1)

- We will use scatterplot to visualize the relationship between Average_Pulse and Calorie_Burnage (we have used the small data set of the sports watch with 10 observations).
- This time we want scatter plots, so we change kind to "scatter":

#Three lines to make our compiler able to draw:

```
import sys
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
health_data = pd.read_csv("data.csv", header=0, sep=",")
```

```
health_data.plot(x='Average_Pulse', y='Calorie_Burnage', kind='scatter'),
```

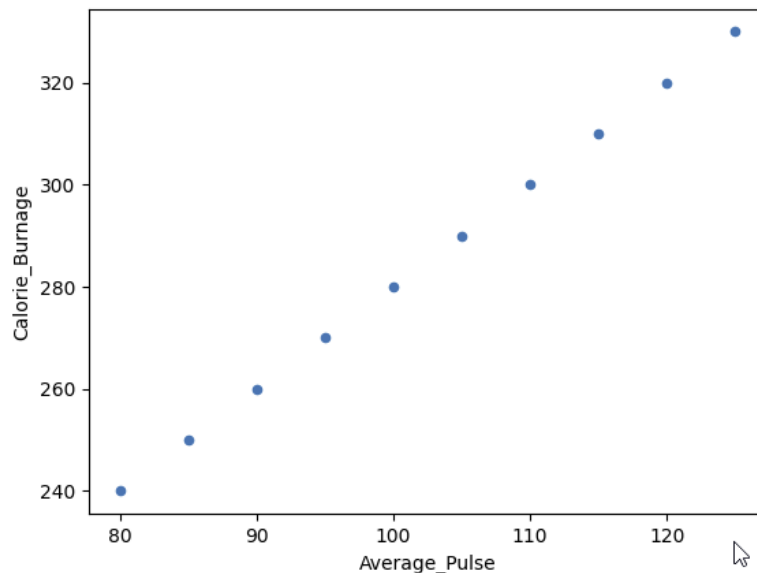
```
plt.show()
```

#Two lines to make our compiler able to draw:

```
plt.savefig(sys.stdout.buffer)
```

```
sys.stdout.flush()
```

output:



Example of a Perfect Negative Linear Relationship (Correlation Coefficient = -1)

#Three lines to make our compiler able to draw:

```
import sys
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
negative_corr = {'Hours_Work_Before_Training': [10,9,8,7,6,5,4,3,2,1],
```

```
'Calorie_Burnage': [220,240,260,280,300,320,340,360,380,400]}
```

```
negative_corr = pd.DataFrame(data=negative_corr)
```

```
negative_corr.plot(x='Hours_Work_Before_Training',y='Calorie_Burnage',  
kind='scatter')
```

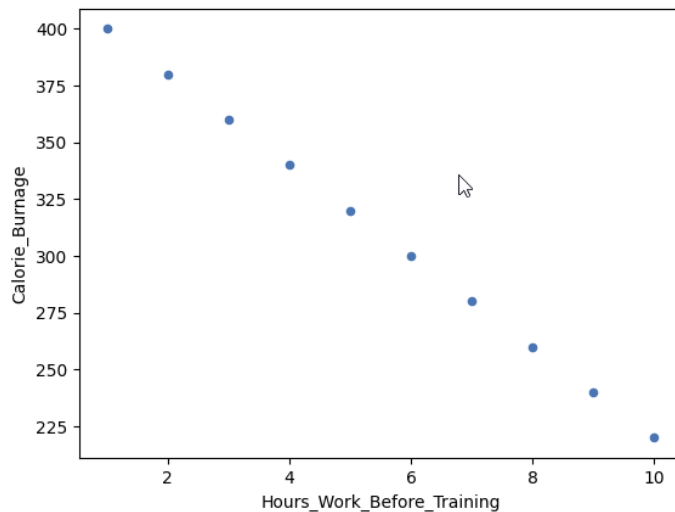
```
plt.show()
```

#Two lines to make our compiler able to draw:

```
plt.savefig(sys.stdout.buffer)
```

```
sys.stdout.flush()
```

output:



We have plotted fictional data here. The x-axis represents the amount of hours worked at our job before a training session. The y-axis is Calorie_Burnage.

If we work longer hours, we tend to have lower calorie burnage because we are exhausted before the training session.

The correlation coefficient here is -1.

Example of No Linear Relationship (Correlation coefficient = 0)

#Three lines to make our compiler able to draw:

```
import sys
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
full_health_data = pd.read_csv("data.csv", header=0, sep=",")
```

```
full_health_data.plot(x='Duration', y='Max_Pulse', kind='scatter')
```

```
plt.show()
```

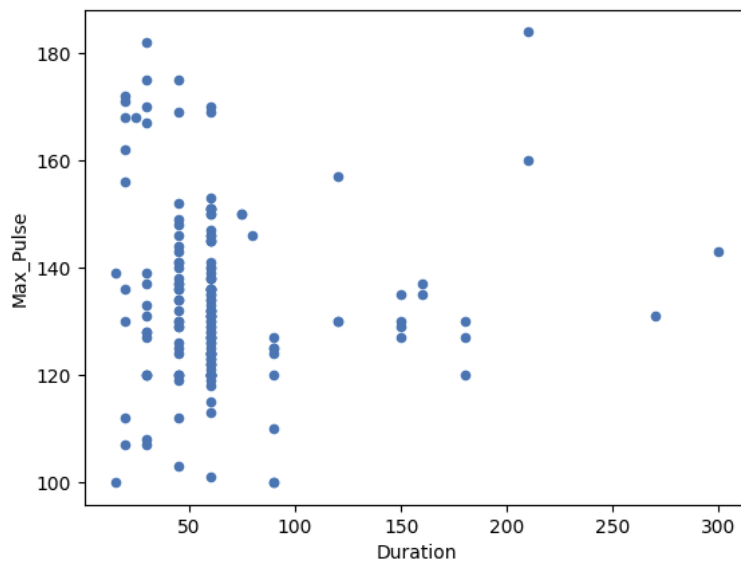
#Two lines to make our compiler able to draw:

```
plt.savefig(sys.stdout.buffer)
```

```
sys.stdout.flush()
```

output:





Population is all elements in a group.

- a **population** is a collection of objects, items (“units”) about which information is sought;

For example,

- College students in US is a population that includes all of the college students in US.
- 25-year-old people in Europe is a population that includes all of the people that fits the description.

It is not always feasible or possible to do analysis on population because we cannot collect all the data of a population. Therefore, we use samples.

Sample is a subset of a population that is observed.

Variables

A variable is an attribute of an object of study that may vary for different cases. Thus, a variable varies for different case studies in research. Considering the previous example of the survey on the temperature of many cities worldwide on the same day, the variables are "Temperature" and "City" because both the attributes vary for different cases.

Now variables can be of two types.

1. **Numerical variable:** They represent values that have numbers. For Example, age, weight, height.

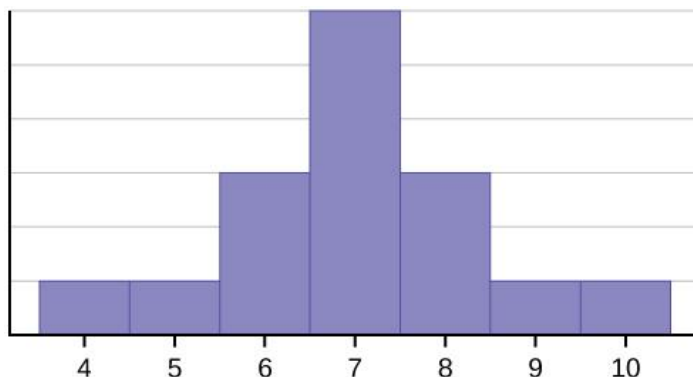
2. **Categorical variable**

These variables represent values that have words, for example, name, nationality, sport, etc. For our previous example of the survey, "Temperature" is a numerical variable.

Consider the following data set.

4; 5; 6; 6; 6; 7; 7; 7; 7; 7; 7; 8; 8; 8; 8; 9; 10

This data set can be represented by following histogram. Each interval has width one, and each value is located in the middle of an interval.



For example,

- 1000 college students in US is a subset of “college students in US” population.

When we compare two samples (or groups), we can use **t-test** to see if there is any difference in means of groups. When we have more than two groups, t-test is not the optimal choice because we need to apply t-test to pairs separately. Consider we have groups A, B and C. To be able to compare the means, we need to apply a t-test to A-B, A-C and B-C. As the number of groups increase, this becomes harder to manage.

In the case of comparing three or more groups, ANOVA is preferred. There are two elements of

ANOVA:

ANOVA stands for analysis of variance and, as the name suggests, it helps us understand and compare variances among groups. Before going in detail about ANOVA, let's remember a few terms in statistics:

- **Mean:** The average of all values.
- **Variance:** A measure of the variation among values. It is calculated by adding up squared differences of each value and the mean and then dividing the sum by the number of samples.

$$Variance = \frac{\sum (x_i - \text{mean})^2}{N}$$

x_i = value i

N = number of values

Mean = average of all values

- **Standard deviation:** The square root of variance.

In order to understand the motivation behind ANOVA, or some other statistical tests, we should learn two simple terms: population and sample.

- Variation within each group
- Variation between groups

ANOVA test result is based on F ratio which is the ratio of the variation between groups to the variation within groups.

$$F = \frac{\text{Variation between groups}}{\text{Variation within groups}}$$

F ratio shows how much of the total variation comes from the variation between groups and how much comes from the variation within groups.

EXERCISES:

Use NumPy to:

- 1) Create an arbitrary one-dimensional array called "**v**".

```
import numpy as np
```

```
v= np.array([3,8,12,18,7,11,30])
```

- 2) Create a new array which consists of the odd indices of previously created array "**v**".

```
odd_elements = v[1::2]
```

3) Create a new array in backwards ordering from v.

```
reverse_order = v[::-1]
```

4) What will be the output of the following code:

```
a = np.array([1, 2, 3, 4, 5])
```

```
b = a[1:4]
```

```
b[0] = 200
```

```
print(a[1])
```

The output will be 200, because slices are views in numpy and not copies.

5) Create a two-dimensional array called "m".

```
m = np.array([ [11, 12, 13, 14], [21, 22, 23, 24], [31, 32, 33, 34] ])
```

6) Create a new array from m, in which the elements of each row are in reverse order.

```
m[:,::-1]
```

7) Another one, where the rows are in reverse order.

```
m[::-1]
```

8) Create an array from m, where columns and rows are in reverse order.

```
m[::-1,::-1]
```

9) Cut of the first and last row and the first and last column

```
m[1:-1,1:-1]
```

10) Define an int16 data type and call this type i16. The elements of the list 'lst' are turned into i16 types to create the two-dimensional array A.

```
import numpy as np
```

```
i16 = np.dtype(np.int16)
```

```
print(i16)
```

```
lst = [ [3.4, 8.7, 9.9], [1.1, -7.8, -0.7], [4.1, 12.3, 4.8] ]
```

```
A = np.array(lst, dtype=i16)
```

```
print(A)
```

```
import numpy as np
```

```
dt = np.dtype([('country', 'S20'), ('density', 'i4'), ('area', 'i4'), ('population', 'i4')])
```

```
population_table = np.array([
```

```
('Netherlands', 393, 41526, 16928800),
```

```
('Belgium', 337, 30510, 11007020),
```

```
('United Kingdom', 256, 243610, 62262000),
```

```
('Germany', 233, 357021, 81799600),
```

```
('Liechtenstein', 205, 160, 32842),
```

```
('Italy', 192, 301230, 59715625),
```

```
('Switzerland', 177, 41290, 7301994),
```

```
('Luxembourg', 173, 2586, 512000),
```

```
('France', 111, 547030, 63601002),
```

```
('Austria', 97, 83858, 8169929),
```

```
('Greece', 81, 131940, 11606813),
```

```
    ('Ireland', 65, 70280, 4581269),
```

```
('Sweden', 20, 449964, 9515744),
```

```
('Finland', 16, 338424, 5410233),
```

```
('Norway', 13, 385252, 5033675)],
```

```
dtype=dt)
```

```
print(population_table[:4])
```

```
In [1]: import numpy as np
dt = np.dtype([('country', 'S20'), ('density', 'i4'), ('area', 'i4'), ('population', 'i4')])
population_table = np.array([
('Netherlands', 393, 41526, 16928800),
('Belgium', 337, 30510, 11007020),
('United Kingdom', 256, 243610, 62262000),
('Germany', 233, 357021, 81799600),
('Liechtenstein', 205, 160, 32842),
('Italy', 192, 301230, 59715625),
('Switzerland', 177, 41290, 7301994),
('Luxembourg', 173, 2586, 512000),
('France', 111, 547030, 63601002),
('Austria', 97, 83858, 8169929),
('Greece', 81, 131940, 11606813),
('Ireland', 65, 70280, 4581269),
('Sweden', 20, 449964, 9515744),
('Finland', 16, 338424, 5410233),
('Norway', 13, 385252, 5033675)],
dtype=dt)
print(population_table[:4])
```

```
[(b'Netherlands', 393, 41526, 16928800)
(b'Belgium', 337, 30510, 11007020)
(b'United Kingdom', 256, 243610, 62262000)
(b'Germany', 233, 357021, 81799600)]
```

```
print(population_table['density'])

print(population_table['country'])

print(population_table['area'][2:5])
```

```
In [2]: print(population_table['density'])
print(population_table['country'])
print(population_table['area'][2:5])
```

```
[393 337 256 233 205 192 177 173 111 97 81 65 20 16 13]
[b'Netherlands' b'Belgium' b'United Kingdom' b'Germany' b'Liechtenstein'
 b'Italy' b'Switzerland' b'Luxembourg' b'France' b'Austria' b'Greece'
 b'Ireland' b'Sweden' b'Finland' b'Norway']
[243610 357021 160]
```

```
In [ ]:
```

12) Attendance of all classes of a school are as follows find their skewness?
1st (35), 2nd(32), 3rd(38), 4th(39), 5th(43)

lass Name	Number of students
1 st	35
2 nd	32
3 rd	38
4 th	39
5 th	45

Q) What does positive skewness mean?

Answer:

Positive skewness means the distribution is skewed to the right. There are more values on the right side of the mean than on the left side.

Condition for Positive Skewness = Mean > median > mode

Q: What does negative skewness mean?

Answer:

Negative skewness means the distribution is skewed to the left. There are more values on the left side of the mean than on the right side.

Condition for negative skewness = Mode > median > mean

Q: What is the formula for calculating skewness?

$$g_1 = \frac{1}{n} \frac{\sum_i (x_i - \mu)^3}{\sigma^3},$$

Chapter Twelve– Regression Analysis

Regression analysis is a predictive modelling technique that assesses the relationship between dependent (i.e., the goal/target variable) and independent factors. Forecasting, time series modelling, determining the relationship between variables, and predicting continuous values can all be done using regression analysis. Just to give you an Analogy, Regression is the best way to study the relationship between household areas and a driver's household electricity cost.

Regression has 2 Categories Namely,

- **Simple Linear Regression:** The association between two variables is established using a straight line in Simple Linear Regression. It tries to create a line that is as near to the data as possible by determining the slope and intercept, which define the line and reduce regression errors. There is a single x and y variable

Equation: $Y = mX + c$

- **Multiple Linear Regression:** Multiple linear regressions are based on the presumption that both the dependent and independent variables, or Predictor and Target variables, have a linear relationship. There are two types of multilinear regressions: linear and nonlinear. It has one or more x variables and one or more y variables, or one dependent variable and two or more independent variables

- Equation: $Y = m_1X_1 + m_2X_2 + m_3X_3 + \dots + c$

- Where,
- Y = Dependent Variable

m = Slope

X = Independent Variable

c = Intercept

- Now, let us understand both Simple and Multiple Linear Regression implementation with the below sample datasets!!

Simple Linear Regression

Given the experience let us predict the salary of the employees using a simple Regression model

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

Read the data from the csv file

```
data = pd.read_csv('/content/Salary_Data.csv') #reading data
```

```
data.head()
```

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

➤ Plot Years of Experience vs Salary based on Experience

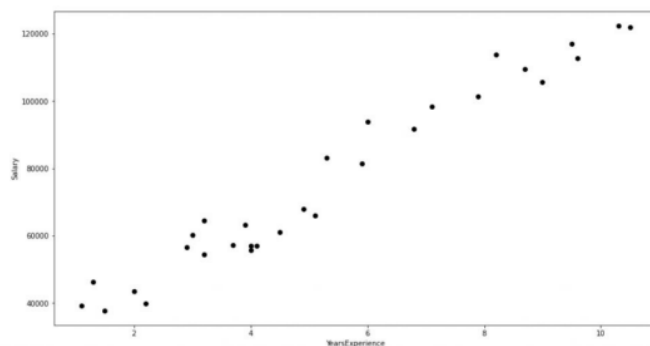
```
plt.figure()
```

```
plt.scatter(data['YearsExperience'],data['Salary'],c='black')
```

```
plt.xlabel("YearsExperience")
```

```
plt.ylabel("Salary")
```

```
plt.show()
```



Reshape and fit the data into simple linear regression

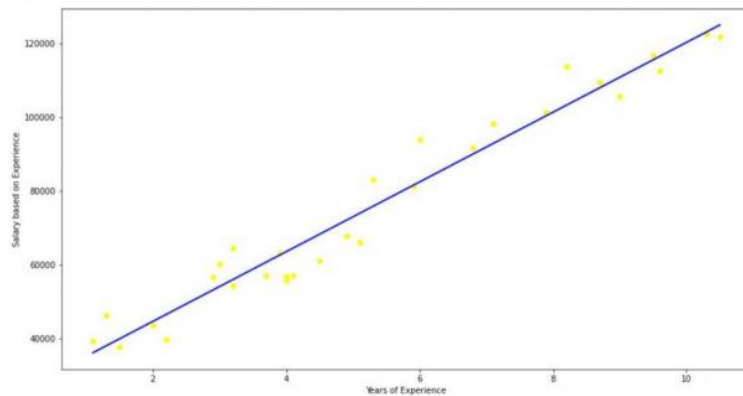
```
X = data['YearsExperience'].values.reshape(-1,1)
```

```
y = data['Salary'].values.reshape(-1,1)
```

```
reg = LinearRegression()
```

```
reg.fit(X, y)
```

Print R-squared value



Multiple Linear Regression

Let us the dataset set of advertising sales based on Tv and Newspaper

Import required libraries

```
%matplotlib inline
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
plt.rcParams['figure.figsize'] = (15.0, 8.0)
```

```
from mpl_toolkits.mplot3d import Axes3D
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error
```

```
from sklearn.metrics import r2_score
```

Read dataset from csv file

```
data = pd.read_csv('/content/Advertising Agency.csv') #reading data
```

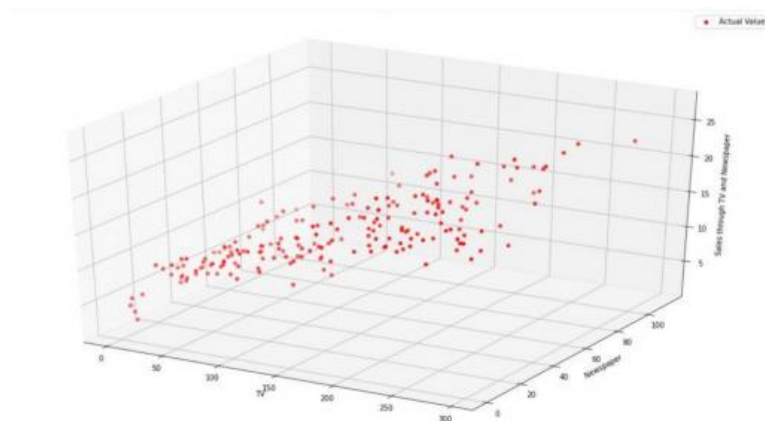
```
data.head()
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

```
tv = data['TV'].values #storing dataframe values as variables
```

```
newspaper = data['Newspaper'].values
```

```
sales = data['Sales'].values
```



Plotting Actual vs Predicted values

```
fig = plt.figure(2)

ax = Axes3D(fig)

ax.scatter(X_train[:,0], X_train[:,1], y_train, color='r',label='Actual Values')

ax.scatter(X_test[:,0],X_test[:,1], y_pred, color='b',label='Predicted Values')

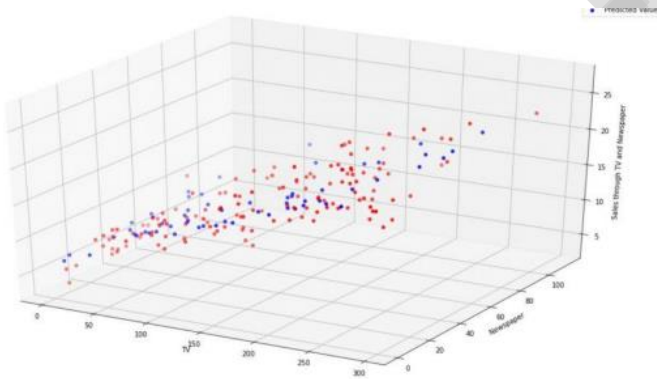
ax.set_xlabel('TV')

ax.set_ylabel('Newspaper')

ax.set_zlabel('Sales through TV and Newspaper')

ax.legend()

plt.show()
```



Multiple regression in machine learning:

Multiple Linear Regression is one of the important regression algorithms which models the linear relationship between a single dependent continuous variable and more than one independent variable.

Example:

Prediction of CO2 emission based on engine size and number of cylinders in a car.

MLR equation:

In Multiple Linear Regression, the target variable(Y) is a linear combination of multiple predictor variables $x_1, x_2, x_3, \dots, x_n$. Since it is an enhancement of Simple Linear Regression, so the same is applied for the multiple linear regression equation, the equation becomes:

$$Y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

Where,

Y= Output/Response variable

$b_0, b_1, b_2, b_3, b_n, \dots$ = Coefficients of the model

$x_1, x_2, x_3, x_4, \dots$ = Various Independent/feature variable

Data Science and Supervised and unsupervised Learning

Bb

Decision Tree

GENERALIZATION ERROR

EVALUATION METRICS

Model Evaluation Metrics define the evaluation metrics for evaluating the performance of a machine learning model, which is an integral component of any data science project. It aims to estimate the generalization accuracy of a model on the future (unseen/out-of-sample) data.

A **confusion matrix** is a matrix representation of the prediction results of any binary testing that is often used to describe the performance of the classification model (or “classifier”) on a set of test data for which the true values are known.

The confusion matrix itself is relatively simple to understand, but the related terminology can be confusing.

		Predicted Values	
		Negative	Positive
Actual Values	Negative	TN True Negative	FP False positive
	Positive	FN False Negative	TP True Positive

Each prediction can be one of the four outcomes, based on how it matches up to the actual value:

- True Positive (TP): Predicted True and True in reality.

- True Negative (TN): Predicted False and False in reality.
- False Positive (FP): Predicted True and False in reality.
- False Negative (FN): Predicted False and True in reality

Now let us understand this concept using hypothesis testing.

A Hypothesis is speculation or theory based on insufficient evidence that lends itself to further testing and experimentation. With further testing, a hypothesis can usually be proven true or false.

A **Null Hypothesis** is a hypothesis that says there is no statistical significance between the two variables in the hypothesis. It is the hypothesis that the researcher is trying to disprove.

We would always reject the null hypothesis when it is false, and we would accept the null hypothesis when it is indeed true.

Even though hypothesis tests are meant to be reliable, there are two types of errors that can occur.

These errors are known as Type 1 and Type II errors.

For example, when examining the effectiveness of a drug, the null hypothesis would be that the drug does not affect a disease.

Type I Error:- equivalent to False Positives(FP).

The first kind of error that is possible involves the rejection of a null hypothesis that is true.

Let's go back to the example of a drug being used to treat a disease. If we reject the null

hypothesis in this situation, then we claim that the drug does have some effect on a disease. But if the null hypothesis is true, then, in reality, the drug does not combat the disease at all. The drug is falsely claimed to have a positive effect on a disease.

Type II Error:- equivalent to False Negatives(FN).

The other kind of error that occurs when we accept a false null hypothesis. This sort of error is

called a type II error and is also referred to as an error of the second kind.

CROSS VALIDATION

Cross validation is a technique for assessing how the statistical analysis generalises to an independent data set. It is a technique for evaluating machine learning models by training several models on subsets of the available input data and evaluating them on the complementary subset of the data. Using cross-validation, there are high chances that we can detect over-fitting with ease.

K-Fold Cross Validation

First I would like to introduce you to a golden rule — “*Never mix training and test data*”. Your first step should always be to **isolate the test data-set** and use it only for final evaluation. Cross-validation will thus be performed on the training set.

Split 1	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 1
Split 2	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 2
Split 3	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 3
Split 4	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 4
Split 5	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Metric 5

Training data

Test data

Initially, the entire training data set is broken up in k equal parts. The first part is kept as the hold out (testing) set and the remaining $k-1$ parts are used to train the model. Then the trained model is then tested on the holdout set. The above process is

repeated k times, in each case we keep on changing the holdout set. Thus, every data point get an equal opportunity to be included in the test set.

Usually, k is equal to 3 or 5. It can be extended even to higher values like 10 or 15 but it becomes extremely computationally expensive and time-consuming. Let us have a look at how we can implement this with a few lines of Python code and the Sci-kit Learn API

```
from sklearn.model_selection import cross_val_score  
  
print(cross_val_score(model, X_train, y_train, cv=5))
```

We pass the **model** or classifier object, the features, the labels and the parameter **cv** which indicates the **K** for K-Fold cross-validation. The method will return a list of k accuracy values for each iteration. In general, we take the average of them and use it as a consolidated cross-validation score.

```
import numpy as np  
  
print(np.mean(cross_val_score(model, X_train, y_train, cv=5)))
```

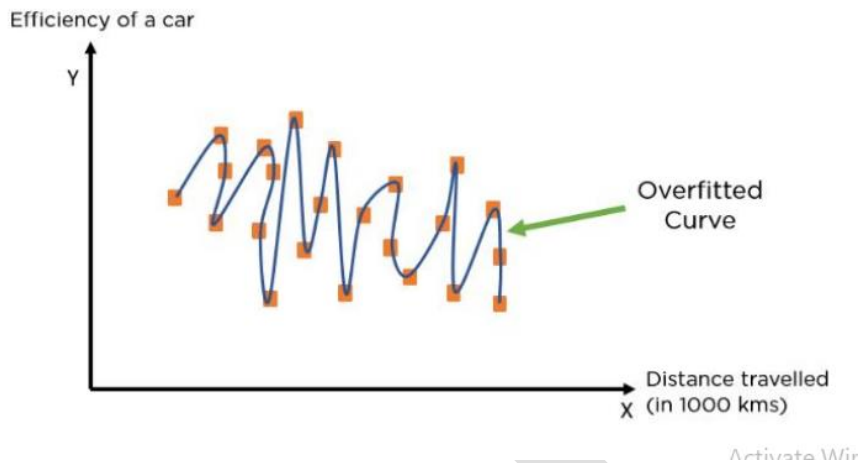
Although it might be computationally expensive, cross-validation is essential for evaluating the performance of the learning model.

Overfitting and Underfitting :

What is Overfitting?

When a model performs very well for training data but has poor performance with test data (new data), it is known as overfitting. In this case, the machine learning model learns the details and noise in the training data such that it negatively affects the performance of the model on test data.

Overfitting can happen due to low bias and high variance.



Reasons for Overfitting

- Data used for **training** is not cleaned and contains noise (garbage values) in it
- The model has a high variance
- The size of the training dataset used is not enough
- The model is too complex

Ways to Tackle Overfitting

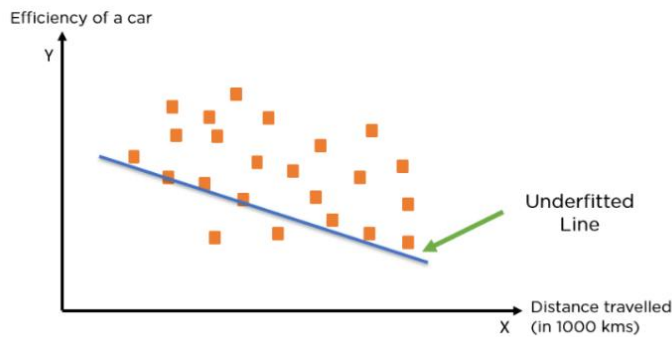
- Using K-fold cross-validation
- Using Regularization techniques such as Lasso and Ridge
- Training model with sufficient data
- Adopting ensembling techniques

What is Underfitting?

When a model has not learned the patterns in the training data well and is unable to generalize

well on the new data, it is known as underfitting. An underfit model has poor performance on the

training data and will result in unreliable predictions. Underfitting occurs due to high bias and low variance.



Reasons for Underfitting

- Data used for training is not cleaned and contains noise (garbage values) in it
- The model has a high bias
- The size of the training dataset used is not enough
- The model is too simple

Ways to Tackle Underfitting

- Increase the number of features in the dataset
- Increase model complexity
- Reduce noise in the data
- Increase the duration of training the data

Ridge Regression

Ridge regression is a model tuning method that is used to analyse any data that suffers from multicollinearity. This method performs L2 regularization. When the issue of multicollinearity occurs, least-squares are unbiased, and variances are large, this results in predicted values being far away from the actual values.

The cost function for ridge regression:

$$\text{Min}(\|Y - X(\theta)\|^2 + \lambda \|\theta\|^2)$$

Lambda is the penalty term. λ given here is denoted by an alpha parameter in the ridge function.

Ridge Regression Models

For any type of regression machine learning model, the usual regression equation forms the base which is written as:

$$Y = XB + e$$

Where Y is the dependent variable, X represents the independent variables, B is the regression coefficients to be estimated, and e represents the errors or residuals.

Ridge Regression Predictions

We now show how to make predictions from a Ridge regression model. In particular, we will make predictions based on the Ridge regression model created for Example 1 with $\lambda = 1.6$.

The raw input data is repeated in range A1:E19 of Figure 1 and the unstandardized regression coefficients calculated in Figure 2 of Ridge Regression Analysis Tool is repeated in range G2:H6 of Figure 1.

	A	B	C	D	E	F	G	H	I	J	K
1	X1	X2	X3	X4	Y			coeff		Pred	Res
2	3	6	2	8	3		Intercept	11.42071		13.48634	-10.4863
3	7	7	11	14	15		X1	0.264684		18.20573	-3.20573
4	11	11	23	33	19		X2	0.315482		22.73489	-3.73489
5	15	12	26	34	27		X3	0.508153		25.42886	1.571137
6	21	16	12	5	23		X4	-0.2047		27.10111	-4.10111
7	23	17	16	10	23					28.95506	-5.95506
8	28	22	22	15	31		MSE	42.68541		33.8813	-2.8813
9	31	16	28	24	39		Lambda	1.6		33.98906	5.010938
10	38	21	34	31	47					39.03526	7.964736
11	39	27	27	8	51					42.34392	8.656083
12	42	24	31	16	47					42.58652	4.413481
13	49	32	40	25	51					49.69422	1.305778
14	57	29	42	21	55					52.70036	2.299637
15	68	36	35	9	63					56.71961	6.280387
16	71	42	39	15	67					60.21096	6.789043
17	89	51	51	23	71					72.27483	-1.27483
18	95	53	60	29	71					77.83906	-6.83906
19	97	55	68	40	75					80.8129	-5.8129
20											554.9103
21	50	20	30	25	41.09159						
22	30	30	20	20	34.89471						

The predictions for the input data are shown in column J. In fact, the values in range J2:J19 can be

calculated by the array formula

=H2+MMULT(A2:D19,H3:H6).

Alternatively, they can be calculated by the array formula

=RidgePred(A2:D19,A2:D19,E2:E19,H9)

Real Statistics Function: The Real Statistics Resource Pack provides the following functions.

RidgeMSE(Rx, Ry, *lambda*) = MSE of the Ridge regression defined by the x data in Rx, y data in Ry and the given lambda value.

RidgePred(Rx0, Rx, Ry, *lambda*): returns an array of predicted y values for the x data in range Rx0 based on the Ridge regression model defined by Rx, Ry and *lambda*; if Rx0 contains only one row then only one y value is returned.

References

- 1- Joel Grus, “Data Science from Scratch First Principals with Python”, 2nd edition, O’REILLY, USA, 2019.
- 2- Laura Lagual and Santi Segui, “**Introduction to Data Science** A Python Approach to Concepts, Techniques and Applications”, Springer 2017

